SET 11

PART A — Short questions and answers

Q1. Classify the relationship "Guest makes Reservation" by its degree and cardinality.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Describe a composite primary key using (Guest_ID, Room_No, Check_In).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Interpret the functional dependency (Guest_ID, Room_No) → Check_In.

Answer: Book_ID → Title means each Book_ID determines exactly one book title.

Q4. Describe the output of a CROSS JOIN between the GUEST and ROOM tables.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe the use of ORDER BY on an aggregate such as SUM(TotalAmount).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Explain the cursor attributes %FOUND and %NOTFOUND.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Explain why a document database suits storing variable room amenities.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Describe the countDocuments() method.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Explain what the following MongoDB query returns: db.rooms.countDocuments({RoomType: "Deluxe"})

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q10. Describe how the RETURN clause selects the properties shown in a Cypher result.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A hotel maintains a Hotel Room Reservation System to store guest details, room details, and reservation

1(a) ER design:

• **GUEST**(GuestID, GuestName, Phone, City); primary key: **GuestID**.

• **ROOM**(RoomNo, RoomType, RatePerNight, Floor); primary key: **RoomNo**.

• **RESERVATION**(ReservationID, GuestID, RoomNo, CheckIn, CheckOut, TotalAmount); primary key: **ReservationID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE GUEST (
  GuestID NUMBER PRIMARY KEY,
  GuestName VARCHAR2(100),
  Phone VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE ROOM (
  RoomNo VARCHAR2(100) PRIMARY KEY,
```

```
  RoomType VARCHAR2(100),
  RatePerNight VARCHAR2(100),
  Floor VARCHAR2(100)
);
CREATE TABLE RESERVATION (
  ReservationID NUMBER PRIMARY KEY,
  GuestID NUMBER,
  RoomNo VARCHAR2(100),
  CheckIn VARCHAR2(100),
  CheckOut VARCHAR2(100),
  TotalAmount VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM RESERVATION x
JOIN GUEST a ON x.GuestID = a.GuestID
JOIN ROOM b ON x.RoomNo = b.RoomNo;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```